PROGRAMACIÓN EN EQUIPO:

-Rodríguez Sánchez Daniel Alexis
-Zuñiga Martinez Edwin

Variante B. Multiplicación de dos dígitos. Leer dos dígitos entre 0 y 9, multiplicarlos y mostrar el producto decimal correcto

- Comandos utilizados para ensamblar y enlazar:

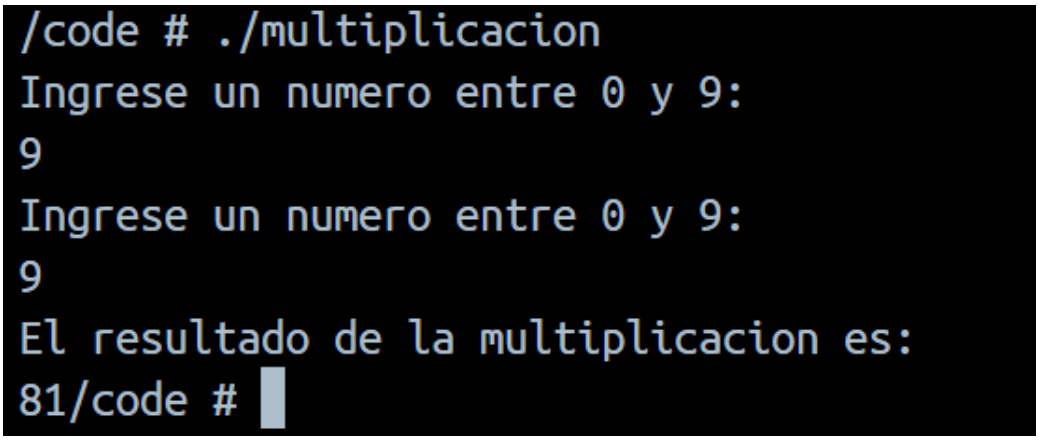
Comando para ensamblar:
nasm -f elf examen.asm

Comando para enlazar:
ld -m elf_i386 -o multiplicacion examen.o

Comando para ejecutar:
./multiplicacion

- Evidencia de cuatro pruebas y sus resultados esperados/obtenidos.

Prueba 1 (Caso límite)
Número 1: 9
Número 2: 9
Resultado esperado: 81
Resultado obtenido:



Prueba 2 (Caso límite)
Número 1: 5
Número 2: 0
Resultado esperado:0
Resultado obtenido:

```
/code # ./multiplicacion
Ingrese un numero entre 0 y 9:
5
Ingrese un numero entre 0 y 9:
0
El resultado de la multiplicacion es:
00/code #
```

Prueba 3
Número 1: 7
Número 2: 6
Resultado esperado: 42
Resultado obtenido:

```
/code # ./multiplicacion
Ingrese un numero entre 0 y 9:
7
Ingrese un numero entre 0 y 9:
6
El resultado de la multiplicacion es:
42/code #
```

Prueba 4
Número 1: 3
Número 2: 2
Resultado esperado: 6
Resultado obtenido:

```
/code # ./multiplicacion
Ingrese un numero entre 0 y 9:
3
Ingrese un numero entre 0 y 9:
2
El resultado de la multiplicacion es:
06/code #
```

Explicación del uso de registros (Entrada, Operación y Salida)

1. Registros utilizados en la Entrada y Salida de datos (int 0x80)
2.

eax: Almacena el código de la llamada al sistema que se va a ejecutar (4 para imprimir mensajes en pantalla con sys_write, 3 para leer datos del teclado con sys_read y 1 para finalizar el programa con sys_exit).

ebx: Define el canal o descriptor de archivo con el que se trabaja (1 para la salida estándar en pantalla stdout, 0 para la entrada estándar por teclado stdin y 0 como código de retorno al cerrar el programa).

ecx: Guarda la dirección de memoria de las etiquetas y variables utilizadas en cada paso (peticion, outMsg, num1, num2 y resultado).

edx: Indica la cantidad exacta de bytes que se van a procesar (lenP y lenout para los mensajes de texto, 5 bytes de lectura para num1 y num2, y 2 bytes para mostrar los dos dígitos guardados en resultado).

2. Registros utilizados en la Operación de Suma y Conversión ASCII
3.

al: Recibe el primer dígito leído desde [num1] y almacena su valor numérico real tras restarle 48. Posteriormente, al ejecutar la instrucción add al, bl, acumula el resultado total de la suma de ambos números. Al dividir entre 10 con div bl, guarda el cociente de la división (el dígito de las decenas), al cual se le suma 48 para convertirlo nuevamente a carácter ASCII y guardarlo en la primera posición de [resultado].

bl: Recibe el segundo dígito leído desde [num2] y guarda su valor numérico tras restarle 48 para sumarlo con al. Una vez realizada la suma, se reutiliza cargándole el valor 10 (mov bl, 10) para actuar como divisor en la instrucción div bl y separar el resultado en dos cifras.

ah: Se asigna en cero (mov ah, 0) justo después de la suma para limpiar la parte alta del registro ax y asegurar que la división de 16 bits entre bl tome únicamente el valor acumulado en al. Al ejecutarse div bl, almacena automáticamente el residuo de la división (el dígito de las unidades), al cual se le suma 48 para convertirlo a ASCII y guardarlo en el segundo byte de memoria [resultado + 1].